

Using ROS

To successfully use ROS, you need to be able to perform basic tasks in Linux. Refer e.g. to [MRS Cheatsheet\[1\]](#) to get a quick reference guide.

Apptainer 101

What is Apptainer

Apptainer is a virtualization technology somewhat similar to Docker. But there are some key differences:

- Apptainer does not fully isolate the containers from the host system.
 - Code running in the container runs under the same user who launched it (as opposed to Docker which usually offers a root user). This also means you do not have `sudo` permissions inside the container if you don't have them in the host system.
 - Code running inside the container has direct access to your home directory and to `/tmp` from the host system. You can add more shared directories by passing the `-B <dir>` CLI option to Apptainer (or by setting the `ARO_MOUNT` environment variable for our launcher script).
 - Code running inside the container has direct access to the host network interfaces and you don't need to do anything to have a working network.
 - Code running inside the container has direct access to the graphical stack of the host so rendering applications work out of the box.
 - There is, however, a catch - most rendering applications use client libraries to do the rendering, and if those differ too much from the host system, there might be compatibility problems.
- Apptainer images can be distributed as a single large file, as opposed to Docker which is usually distributed via online registries.
- Apptainer does not know the concept of image layers.
- Containers run in Apptainer have no "runtime presence". You do not need to start and stop them explicitly. The container lives as long as the program running inside it.
- Apptainer images can be converted from Docker images.
- Apptainer images are immutable. You cannot change or update the system in the image (unless using overlays, which is way beyond the scope of ARO).

We choose Apptainer over Docker because it is friendlier with [GUI](#) applications and it is the framework used in HPC (High-performance Computing) clusters such as the RCI cluster running at CTU, or Metacentrum. Also, running Apptainer containers doesn't require administrator privileges. We also used Apptainer with success for running local multi-robot simulation tests for DARPA SubT Challenge.

Apptainer vs. Singularity

[Singularity CE\[2\]](#) == [Apptainer\[3\]](#) (the latter is just a newer name). You can find both terms when googling.

We use Apptainer, but Singularity would also work. It just uses `singularity` wherever we use `apptainer`.

At Lab Computers / Turtlebots

At [labs\[4\]](#) (e.g., E130, E132) and the Turtlebot robots, Robot Operating System (ROS) is available through pre-downloaded [Apptainer\[5\]](#) containers. The computers and robots automatically handle updating the images so you can be sure you're always using the latest image.

A general guide for connecting to the desktops/robots in the lab is here: [TurtleBot Lab Guide\[6\]](#). Use your CTU password to login to these machines.

To start using ROS in Apptainer, clone the [deploy repository\[7\]](#) somewhere to your home directory. Once you have the deploy repository, you can run our Apptainer image with Ubuntu 24.04 + ROS Jazzy using: `scripts/start_apptainer_aro`.

First-time run:

```
git clone https://gitlab.fel.cvut.cz/robofab/deploy.git
deploy/scripts/start_apptainer_aro
# console prompt should start with ARO now
```

Following runs:

```
deploy/scripts/start_apptainer_aro
# console prompt should start with ARO now
```

The script will create (or update) a ROS workspace in folder `workspace` next to `deploy`. The codes you need to edit for homeworks are all in `workspace/src/student-packages`.

You can make use of the fact that the home directory in labs is a shared network mount, so changes you do on the desktop computer are almost instantly available also on the turtlebots.

On Your Computer

You will get the smoothest and easiest experience on Ubuntu 24.04. Newer Ubuntu's still offer a smooth experience. Other Linux distros can be used, but may require a few manual setup steps. Apptainer only runs on Linux. If you need to use a different operating system, there are alternatives, which are, however, not officially supported by the teaching team[8]. We think it is good for you to get an Ubuntu installation and familiarize yourself with it during the study of Cybernetics.

If you have Ubuntu XY.10 or ZZ.04 with odd ZZ, you are running a short-term support version. These versions lose support quite quickly (only 9 months of support since release). You can see errors like `E: The repository ... does not have a Release file. N: Updating from such a repository can't be done securely, and is therefore disabled by default. N: See apt-secure(8) manpage for repository creation and user configuration details.` . This means your system is no longer supported and you should ideally update it to a newer (LTS) version.

If you don't have anything important on your Linux install, just reinstall it. If you have important data on it, you can try to just get Apptainer installed via alternative ways, like building it from source. That would allow you to run ARO Apptainer even on this unsupported system. First time, choose a folder on your computer that has enough disk space (at least 3 GB) and is preferably on an SSD. `cd` to the folder and run the following commands that will download the deploy repository, install Apptainer, download the image and finally start a container.

The `install_apptainer` script works on Ubuntu 24.04+. If you have a different Linux flavor, you will need to install Apptainer manually. The `install_apptainer` script will point you to the relevant documentation.

```
git clone https://gitlab.fel.cvut.cz/robofab/deploy.git
deploy/scripts/install_apptainer
deploy/scripts/download_apptainer_image
deploy/scripts/start_apptainer_aro
# console prompt should start with ARO now
```

The script will create (or update) a ROS workspace in folder `workspace` next to `deploy` . The codes you need to edit for homeworks are all in `workspace/src/student-packages/aro_exploration` .

Once you have the initial installation done, running Apptainer with ROS is as easy as:

```
deploy/scripts/start_apptainer_aro
# console prompt should start with ARO now
```

The script will automatically check for new releases of the Apptainer ARO image and download them if necessary.

Using IDEs with Apptainer

By default, Apptainer makes your normal home directory accessible inside the container, so you can edit files using any editor you like. However, to get a fully-featured IDE experience, it is better to start an IDE from within the container, as then it has access to all the ROS programs that are needed to test your code. The provided Apptainer image comes with several IDEs preinstalled:

```
code . # Start VSCode inside the Apptainer container
pycharm # Start Pycharm inside the Apptainer container
subl # Start Sublime editor inside the Apptainer container
vim # start VIM
nvim # start neovim
```

Updates of student-packages repository

The `start_apptainer_aro` script will also auto-update the copy of student-packages repository where you work on your homeworks. We expect the updates of this repository should not be frequent, but we reserve the right to push updates during the semester. The automatic update works by calling `git fetch origin && git merge origin/master` in the repository. If you only edit the parts of files you are supposed to, the update should be seamless. If your local changes are not compatible with the update, you will be informed by the script and it will guide you through the next possible steps.

Behind the Scenes

For the curious, here's a short summary what all the provided scripts do for you:

- Check that Apptainer is installed and install it if possible from PPA <https://launchpad.net/~apptainer/+archive/ubuntu/ppa>[9] .
- Autoupdate the deploy repository to pull new fixes.
- Download the latest ARO Apptainer image from <https://gitlab.fel.cvut.cz/robolab/deploy/>[10] choosing the correct CPU architecture.
- Run `apptainer exec robolab.sif initialize_workspace_aro` . Script `initialize_workspace_aro` running inside the Apptainer container does the following:
 - Create the necessary folder structure
 - Clone or update <https://gitlab.fel.cvut.cz/robolab/aro/student-packages>[11]
 - Initiaize the colcon workspace
 - Build the workspace using `colcon build` , but only once, as we only use Python packages (with C++ nodes, we would need to build after every change).
 - Source the workspace using `source ~/workspace/install/setup.bash` to activate it in the current console.

Common problems

- If you have automatic activation of Conda base env in your `.bashrc` file, it will most probably break ROS. You can see errors like

```
ImportError: "from catkin_pkg.package import parse_package" failed: No module
```

- To fix this, either remove the automatic conda activation, or make it conditional to not run inside Apptainer. E.g.

```
if [ -z "$APPTAINER_NAME" ]; then ORIGINAL_CONDA_ACTIVATION_CODE; fi
```

- If you see errors about missing Numpy functions, you probably have a pip-installed Numpy that is too new. This can happen if you e.g. `pip3 install --user --upgrade numpy`. Never do this on Ubuntu. pip was not designed to manage distribution-wide Python packages and doing this can break your system (pip now also says this warning). Fix this by `pip3 uninstall numpy`. If numpy is broken after this, call `sudo apt install --reinstall python3-numpy`.
- ROS packages `aro_exploration` is not found by `ros2 launch`. You probably source some ROS workspaces in your `~/.bashrc`. Make that sourcing conditional, i.e. move the sourcing into a block like

```
if [ -z "$APPTAINER_NAME" ]; then YOUR_ROS_SOURCING_COMMANDS; fi
```

- Also make sure you have the following code block in your `~/.profile`:

```
if [ -n "$BASH_VERSION" ]; then
    # include .bashrc if it exists
    if [ -f "$HOME/.bashrc" ]; then
        . "$HOME/.bashrc"
    fi
fi
```

- Command completion using `<Tab>` doesn't work in tmux: Add the following in your `.bashrc`:

```
[ -n "$APPTAINER_NAME" ] && [ -n "$TMUX" ] && source /path/to/your/workspace/i
```

- Running ROS code shows you this warning: `[rmw_zenoh_cpp]: Unable to connect to a Zenoh router. Have you started a router with `ros2 run rmw_zenoh_cpp rmw_zenohd`?`
 - Okay, do what it suggests, launch zenohd!
- Robot model does not appear in RViz, the simulation is not running and command `gz topic -l` does not show any topics while showing no errors in the command line. The Gazebo GUI-

server communication can be blocked by firewall. Check whether firewall is active on your computer and adapt its settings according to [this github issue\[12\]](#).

- When you start a launch file and it freezes quite early after printing only a few lines, you might have a problem with `localhost` name resolution. Open `/etc/hosts` with a text editor and see if line with `127.0.0.1` contains `localhost`. If not, add `localhost` to this line (`sudo nano /etc/hosts`). You can also test this problem with `ping localhost` (outside of the Apptainer container).

Hypertext Links

[1] https://raw.githubusercontent.com/ctu-mrs/mrs_cheatsheet/gh-pages/mrs_cheatsheet.pdf

[2] <https://sylabs.io/singularity/>

[3] <https://apptainer.org/>

[4] <https://cyber.felk.cvut.cz/study/computer-labs/>

[5] [https://en.wikipedia.org/wiki/Singularity_\(software\)](https://en.wikipedia.org/wiki/Singularity_(software))

[6] <https://cw.fel.cvut.cz/wiki/courses/aro/tutorials/turtlebots>

[7] <https://gitlab.fel.cvut.cz/robolab/deploy>

[8] https://cw.fel.cvut.cz/wiki/courses/aro/tutorials/alternative_install

[9] <https://launchpad.net/~apptainer/+archive/ubuntu/ppa>

[10] <https://gitlab.fel.cvut.cz/robolab/deploy/>

[11] <https://gitlab.fel.cvut.cz/robolab/aro/student-packages>

[12] <https://github.com/gazebo-sim/gz-sim/issues/2917>

[courses/aro/tutorials/ros.txt](#) · Last modified: 2026/03/23 13:44 by peckama2

Copyright © 2026 CTU in Prague | Operated by [IT Center of Faculty of Electrical Engineering](#)
| Bug reports and suggestions [ServiceDesk CTU](#)